Light

CPU Architecture & Von Neumann

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Von Neumann architecture
2. CPU components
3. Performance factors
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **CPU Architecture (1CP2/01 Topic 3: Computers)**. This topic covers the internal structure and operation of the Central Processing Unit (CPU), including the Von Neumann architecture, the roles of the ALU, control unit, registers and buses, and factors affecting CPU performance such as clock speed, cache size and number of cores.

Exam questions typically ask you to **identify** CPU components, **describe** their purposes, **explain** how they work together, and **discuss** factors affecting performance. The 2024 Paper 1 included a diagram-labelling question on buses and clock connections. Expect questions worth 2—6 marks that require you to link component functions to the fetch-decode-execute cycle.

Why CPU architecture matters

The CPU is the "brain" of a computer system. Understanding its architecture helps you:

- **Understand how programs run:** Every instruction your code executes passes through the CPU's fetch-decode-execute cycle
- **Make informed hardware choices:** Knowing what clock speed, cores and cache mean helps when selecting or comparing computers

- **Write efficient code:** Understanding CPU limitations can help you write programs that run faster
- **Debug hardware issues:** Recognising what each component does helps diagnose problems
- **Succeed in the exam:** CPU architecture appears in Paper 1 almost every year, typically worth 4—8 marks
  In the exam, you will be asked to label diagrams, describe component functions, and explain how performance factors affect processing speed. The key is understanding how components work **together**, not just in isolation.

What you must know

This checklist covers the key exam-visible concepts from the Edexcel specification (3.1.1). You should be able to explain each point and apply it to exam scenarios.

- **Define** what is meant by the Von Neumann architecture and state its key characteristic (stored program concept)
- **Identify** the main components of the CPU: Arithmetic Logic Unit (ALU), Control Unit (CU), and Registers
- **Describe** the purpose of CPU registers in the context of the fetch-decode-execute cycle
- **Explain** the role of each bus: address bus (unidirectional), data bus (bidirectional), and control bus (bidirectional)
- **Describe** the role of the system clock in synchronising CPU operations
- **Calculate** the number of addressable memory locations using 2^n where n = address bus width

> **Specification link:** This topic covers Edexcel 1CP2 specification section 3.1.1 — "understand the von Neumann stored program concept and the role of main memory (RAM), CPU (control unit, arithmetic logic unit, registers), clock, address bus, data bus, control bus in the fetch-decode-execute cycle."

Toolkit

These are the key patterns, definitions and relationships you need to apply in exam questions.

CPU component summary

The Edexcel spec mentions "CPU (control unit, arithmetic logic unit, registers)" — the specific registers below are commonly taught to help explain the FDE cycle.

Component	Full name	Purpose (exam wording)
ALU	Arithmetic Logic Unit	Performs arithmetic operations (add, subtract) and logical operations (AND, OR, NOT, comparisons)
CU	Control Unit	Controls and coordinates the activities of the CPU; decodes instructions; sends control signals
Clock	System Clock	Generates regular pulses to synchronise CPU operations; sends signals to the Control Unit

Component	Full name	Purpose (exam wording)
PC	Program Counter	Holds the memory address of the next instruction to be fetched
MAR	Memory Address Register	Holds the memory address of data or an instruction to be fetched or written
MDR	Memory Data Register	Holds the data or instruction that has been fetched from memory or is waiting to be written
Accumulator	Accumulator	Stores the results of calculations performed by the ALU

Bus summary

Bus	Direction	Purpose	Width affects...
Address bus	Unidirectional (CPU → memory)	Carries the memory address of data to be read or written	Maximum addressable memory (2^n locations)
Data bus	Bidirectional	Carries data between the CPU and memory/peripherals	Amount of data transferred per operation
Control bus	Bidirectional	Carries control signals (read, write, clock, interrupt)	Types of operations that can be signalled

Performance factors formula patterns

Clock speed: Measured in Hz (cycles per second). Higher clock speed = more instructions processed per second.
Example: A 3.0 GHz processor can perform 3,000,000,000 (3 billion) clock cycles per second.
Address bus width: Number of unique memory addresses = 2^n where n = number of address lines.
Example: A 32-bit address bus can address 2^{32} = 4,294,967,296 memory locations (4 GiB).

Key relationships to remember

- **Clock → Control Unit:** The clock sends regular pulses to the CU to synchronise operations
- **PC → MAR:** The address in the PC is copied to the MAR during the fetch stage
- **Memory → MDR:** Data fetched from RAM is placed in the MDR

- **MDR → CU:** Instructions in the MDR are sent to the CU for decoding
- **ALU → Accumulator:** Results of calculations are stored in the accumulator

1. Von Neumann architecture

The **Von Neumann architecture** is the fundamental design used by most modern computers. It was proposed by mathematician John von Neumann in 1945 and describes how a computer's hardware is organised to store and process instructions.

1.1 The stored program concept

The key idea of Von Neumann architecture is the **stored program concept**: both the program instructions and the data they operate on are stored together in the same main memory (RAM).

Before Von Neumann, early computers had to be physically rewired to run different programs. The stored program concept meant that programs could be loaded from storage and changed without modifying the hardware.

Key characteristics:

- Instructions and data share the same memory
- Instructions are fetched and executed sequentially (one at a time)
- A single set of buses connects the CPU to memory
- The CPU processes one instruction at a time (single processor)

1.2 Components of Von Neumann architecture

A Von Neumann system consists of:

- **Central Processing Unit (CPU):** Contains the ALU, CU, and registers
- **Main memory (RAM):** Stores instructions and data currently in use
- **Input devices:** Allow data to enter the system (keyboard, mouse, sensors)
- **Output devices:** Display or transmit results (monitor, printer, speakers)
- **Buses:** Communication pathways connecting all components

> **Exam focus:** When asked to "describe the Von Neumann architecture", always mention the stored program concept — this is the defining feature that distinguishes it from earlier computer designs.

1.3 Von Neumann diagram

```
┌──────────────────────────────────────────────────────────┐ │ CPU │ │ ┌──────────┐
┌──────────────────────┐ │ │ │ CU │ │ Registers │ │ │ │ (Control │ │ PC MAR
MDR ACC │ │ │ │ Unit) │ └──────────────────────┘ │ │ └──────────┘ │ │
┌──────────┐ ┌──────────────────────┐ │ │ │ ALU │ │ Cache │ │ │ └──────────┘
└──────────────────────┘ │ └──────────────────────────────────────────────┘ │
│ │ ┌──────┐ ┌──────┐ ┌──────┐ │ Address │ │ Data │ │ Control
│ │ Bus │ │ Bus │ │ Bus │ └──────┘ └──────┘ └──────┘ │ │ │
└──────────┴──────────┴──────────┘ │ ┌──────────────────────┐ │ Main Memory │ │
(RAM) │ │ Instructions and │ │ Data │ └──────────────────────┘
```

This diagram shows the key components of Von Neumann architecture. The three buses connect the CPU to main memory, allowing instructions and data to be transferred.

1.4 Advantages and limitations

Advantages:

- Simple, well-understood design

- Programs can be easily changed without hardware modification
- Same memory used for instructions and data (efficient use of resources)

Limitations:

- **Von Neumann bottleneck:** The CPU can only fetch one instruction or piece of data at a time via the shared bus, limiting processing speed
- Sequential processing is slower than parallel processing

Worked Example 1 – Defining Von Neumann architecture

Question: State what is meant by the stored program concept in Von Neumann architecture. [2 marks]

Model answer:
The stored program concept means that program instructions and data are stored together in the same main memory **(1)**.
This allows programs to be changed without modifying the hardware / instructions can be fetched and executed sequentially from memory **(1)**.

Exam tip: For 2 marks, give the definition (instructions + data in same memory) AND a consequence or benefit (programs can be changed easily, sequential execution, etc.).

Worked Example 2 – Identifying Von Neumann components

Question: A computer uses Von Neumann architecture. Identify two components found inside the CPU. [2 marks]

Model answer:
Arithmetic Logic Unit (ALU) **(1)**
Control Unit (CU) **(1)**
Also acceptable: Cache, Program Counter, MAR, MDR, Accumulator, Registers

Exam tip: Read carefully — the question asks for components **inside the CPU**. RAM and buses are NOT inside the CPU. Common error: listing "memory" as a CPU component.

Worked Example 3 – Explaining the Von Neumann bottleneck

Question: Explain one limitation of Von Neumann architecture. [2 marks]

Model answer:
The Von Neumann bottleneck occurs because instructions and data share the same bus **(1)**.

This means the CPU cannot fetch an instruction and data at the same time, which limits processing speed **(1)**.

Exam tip: "Explain" requires you to give a reason or consequence. State the limitation AND explain why it causes a problem.

Worked Example 4 — Comparing stored program to earlier designs

Question: Before Von Neumann architecture, computers had to be physically rewired to run different programs. Explain one benefit of the stored program concept. [2 marks]

Model answer:
Programs can be changed by loading new instructions into memory **(1)**.
This is faster and more flexible than rewiring hardware, allowing the same computer to run many different programs **(1)**.

Exam tip: Link your answer to the context given in the question. Here, compare to "physically rewired" — the benefit is that you don't need to do that anymore.

2. CPU components

The CPU contains several specialised components that work together to fetch, decode and execute instructions. Understanding the role of each component is essential for exam success.

2.1 The Arithmetic Logic Unit (ALU)

The **Arithmetic Logic Unit (ALU)** is the part of the CPU that performs calculations and makes decisions.

Arithmetic operations:

- Addition (+)
- Subtraction (−)
- Multiplication (×)
- Division (÷)

Logical operations:

- AND, OR, NOT, XOR (Boolean logic)
- Comparisons: equal to, greater than, less than
- Bit shifting (left shift, right shift)

The results of ALU operations are typically stored in the **accumulator** register.

2.2 The Control Unit (CU)

The **Control Unit (CU)** is the "manager" of the CPU. It coordinates all activities and ensures instructions are executed in the correct order.

Functions of the CU:

- **Fetches** instructions from memory
- **Decodes** instructions to determine what operation is needed
- **Sends control signals** to other components (ALU, memory, I/O devices)
- **Controls timing** of operations using the system clock
- **Manages data flow** between CPU components and memory

> **Exam tip:** The CU does NOT perform calculations — that's the ALU's job. The CU **controls** and **coordinates**. If asked "what does the CU do?", use words like "control", "coordinate", "decode", "send signals".

2.3 Registers

Registers are tiny, extremely fast memory locations inside the CPU. They temporarily hold data, addresses and instructions during processing.

Program Counter (PC)

Holds the **memory address of the next instruction** to be fetched. After each fetch, the PC is incremented to point to the following instruction (unless a jump/branch occurs).

Memory Address Register (MAR)

Holds the **memory address** of data or an instruction that is about to be fetched from or written to memory. The MAR is connected to the address bus.

Memory Data Register (MDR)

Also called the Memory Buffer Register (MBR). Holds the **actual data or instruction** that has been fetched from memory, or data waiting to be written to memory. The MDR is connected to the data bus.

Accumulator (ACC)

Stores the **results of calculations** performed by the ALU. Acts as a temporary working area for arithmetic and logical operations.

2.4 The system clock

The **system clock** generates regular electrical pulses that synchronise all CPU operations. Each pulse is called a **clock cycle**, and the clock ensures that all components work together in time.

- **Synchronisation:** The clock sends pulses to the Control Unit to coordinate the fetch-decode-execute cycle
- **Timing:** Each stage of the FDE cycle happens on a clock pulse
- **Clock speed:** Measured in Hertz (Hz) — modern CPUs run at several GHz (billions of cycles per second)

> **Key point:** The clock signal goes TO the Control Unit. In diagrams, always draw the arrow from Clock → CU (unidirectional). This was tested in the 2024 exam.

2.5 Buses

Buses are sets of parallel wires that carry signals between CPU components and memory. There are three main buses in a Von Neumann system.

Address Bus

Direction: Unidirectional (CPU → memory only)

Data Bus

Direction: Bidirectional (CPU ↔ memory)

Purpose: Carries the memory address of data to be read or written
Width matters: An n-bit address bus can address 2^n memory locations
Example: 32-bit address bus = 2^{32} = 4 billion addresses

Purpose: Carries data between CPU and memory/devices
Width matters: Determines how much data can be transferred in one operation
Example: 64-bit data bus = 8 bytes transferred per cycle

Control Bus
Direction: Bidirectional
Purpose: Carries control signals (read, write, clock pulses, interrupts)
Signals include: Memory read, memory write, bus request, interrupt request, clock

Worked Example 5 — Describing register functions
Question: Describe the purpose of the Program Counter (PC) and the Memory Address Register (MAR). [4 marks]

Model answer:
Program Counter (PC):
The PC holds the memory address of the next instruction to be fetched **(1)**.
After each instruction is fetched, the PC is incremented to point to the next instruction in sequence **(1)**.
Memory Address Register (MAR):
The MAR holds the memory address of data or an instruction that is to be fetched or written **(1)**.
The contents of the MAR are sent to memory via the address bus **(1)**.

Exam tip: For register questions, state what the register **holds** and what **happens** to it or how it is **used**. Two points per register = full marks.

Worked Example 6 — Explaining bus directions
Question: Explain why the address bus is unidirectional but the data bus is bidirectional. [4 marks]

Model answer:
Address bus (unidirectional):
The CPU sends memory addresses to locate data in RAM **(1)**.

Memory does not need to send addresses back to the CPU, so the bus only goes one way **(1)**.
Data bus (bidirectional):
Data needs to travel from memory to the CPU when reading **(1)**.
Data also needs to travel from the CPU to memory when writing, so the bus must go both ways **(1)**.

Exam tip: "Explain why" questions require reasoning. Don't just state the direction — explain what travels on the bus and why that determines the direction.

Worked Example 7 – Calculating addressable memory

Question: A computer has a 16-bit address bus. Calculate the maximum number of memory locations that can be addressed. [2 marks]

Step 1: Identify the formula: Number of addresses = 2^n where n = address bus width
Step 2: Substitute: 2^{16} **(1)**
Answer: 65,536 memory locations **(1)**
(Also acceptable: 64 Ki locations, or 64 × 1024)

Exam tip: Know your powers of 2! 2^{10} = 1024, 2^{16} = 65,536, 2^{20} = 1,048,576 (≈1 million), 2^{32} ≈ 4 billion.

Worked Example 8 – Describing the ALU and CU

Question: A CPU contains an Arithmetic Logic Unit (ALU) and a Control Unit (CU). Describe the purpose of each. [4 marks]

Model answer:
Arithmetic Logic Unit (ALU):
The ALU performs arithmetic operations such as addition, subtraction, multiplication and division **(1)**.
It also performs logical operations such as AND, OR, NOT and comparisons **(1)**.
Control Unit (CU):
The CU decodes instructions and controls the operation of other CPU components **(1)**.
It sends control signals to coordinate the movement of data between components and manages the timing of operations **(1)**.

> **Exam tip:** Give specific examples of what each component does. "Performs calculations" is vague — "performs addition and subtraction" is specific. "Controls things" is vague — "sends control signals" is specific.

3. How CPU components work together

Understanding how the CPU components interact during the fetch-decode-execute cycle is essential. This section explores these relationships and includes some enrichment material on factors that affect CPU performance.

3.1 The role of the clock

The **system clock** generates regular electrical pulses that synchronise all CPU operations. Without the clock, the CPU components would not work in coordination.

Key points about the clock:

- The clock sends pulses TO the Control Unit
- Each pulse triggers the next step in the FDE cycle
- The clock ensures all components work in time with each other
- Clock speed is measured in Hertz (Hz) — cycles per second

Units: 1 MHz = 1 million cycles/second. 1 GHz = 1 billion cycles/second. Modern CPUs run at several GHz.

3.2 Component interactions during FDE

During the fetch-decode-execute cycle, the CPU components work together in a precise sequence:

- **PC → MAR:** The address of the next instruction is copied from the Program Counter to the Memory Address Register
- **MAR → Address bus:** The address is sent to memory via the address bus
- **Memory → MDR:** The instruction/data is fetched and placed in the Memory Data Register via the data bus
- **MDR → CU:** The instruction is sent to the Control Unit for decoding
- **CU → ALU:** If the instruction requires a calculation, the CU sends signals to the ALU
- **ALU → Accumulator:** The result of the calculation is stored in the Accumulator

> **Exam tip:** When asked how components "work together" or "interact", describe the flow of data between them. Use arrows (→) to show direction. Always mention which bus is used for each transfer.

3.3 Diagram labelling skills

Exam questions often ask you to label CPU diagrams. The 2024 Paper 1 included exactly this type of question. Key things to remember:

Address bus: Unidirectional arrow (CPU → Memory)
Data bus: Bidirectional arrow (CPU ↔ Memory)
Control bus: Bidirectional arrow
Clock: Unidirectional arrow (Clock → Control Unit)

Remember: The clock sends signals TO the Control Unit. This is a common exam question — always draw the arrow pointing FROM the clock TO the CU.

3.4 Address bus width calculations

The width of the address bus determines how many memory locations can be accessed:

Formula: Number of addresses = 2^n
where n = number of bits in the address bus

Examples:

- 8-bit address bus: 2^8 = 256 addresses
- 16-bit address bus: 2^{16} = 65,536 addresses
- 32-bit address bus: 2^{32} = 4,294,967,296 addresses (≈4 billion)

This was tested in the 2024 Paper 1: "The address bus of a computer is 8-bits wide. State the number of unique addresses that can be accessed."

Beyond the Edexcel spec — Enrichment only: The following content on CPU performance factors (clock speed comparisons, multiple cores, cache memory) provides useful background knowledge but goes beyond what is explicitly required by the Edexcel 1CP2 specification. This material may help deepen your understanding but is unlikely to appear as a direct exam question.

Enrichment: Factors affecting CPU performance

In real-world computing, several factors affect how fast a CPU can process instructions:

Clock speed

Higher clock speed = more cycles per second = faster processing
Trade-off: More heat generated, more power consumed

Number of cores

More cores = parallel processing = multiple tasks at once
Trade-off: Software must be designed to use multiple cores

Cache memory

Cache is fast memory inside the CPU that stores frequently used data
Trade-off: Cache is expensive and small in size

Worked Example 9 — Explaining component interactions

Question: Describe how the Program Counter (PC) and Memory Address Register (MAR) work together during the fetch stage of the FDE cycle. [3 marks]

Model answer:

The Program Counter holds the memory address of the next instruction to be fetched **(1)**.
This address is copied from the PC to the Memory Address Register **(1)**.
The MAR then sends this address to memory via the address bus so the instruction can be retrieved **(1)**.

Exam tip: "Work together" questions want you to show the FLOW between components. Use words like "copied to", "sent to", "transferred via" to show the connection.

Worked Example 10 — Calculating addressable memory (2024 exam style)

Question: The address bus of a computer is 8-bits wide. State the number of unique addresses that can be accessed. [1 mark]

Model answer:
2^8 = 256 unique addresses **(1)**
Also acceptable: 2 × 2 × 2 × 2 × 2 × 2 × 2 × 2

Exam tip: This exact question appeared in the 2024 Paper 1! Learn your powers of 2: 2^8 = 256, 2^{10} = 1024, 2^{16} = 65,536.

Worked Example 11 — Describing ALU and accumulator relationship

Question: Explain the relationship between the ALU and the accumulator during the execute stage. [2 marks]

Model answer:
The ALU performs arithmetic and logical operations on data **(1)**.
The results of these calculations are stored in the accumulator register **(1)**.

Exam tip: The accumulator is the ALU's "output register". After any calculation, ask yourself: "Where does the result go?" Answer: the accumulator.

Worked Example 12 — Labelling a CPU diagram

Question: The diagram shows part of a CPU connected to main memory. Complete the diagram by:

- Labelling the address bus correctly
- Labelling the data bus correctly

- Drawing an arrow from the clock to the control unit, showing the correct direction [3 marks]

Model answer:
Address bus: Label the bus that connects CPU to memory with a unidirectional arrow (CPU → Memory) **(1)**
Data bus: Label the bus that connects CPU to memory with a bidirectional arrow (CPU ↔ Memory) **(1)**
Clock to CU: Draw an arrow FROM the clock TO the control unit (unidirectional: Clock → CU) **(1)**

Exam tip: Direction matters! The clock sends signals TO the CU (not the other way around). The address bus is UNIDIRECTIONAL from CPU to memory. The data bus is BIDIRECTIONAL. Practice labelling diagrams before the exam.

Worked examples summary

The table below summarises the 12 worked examples covered in this topic, organised by section and difficulty level.

#	Topic	Question type	Marks	Key skill
1	Von Neumann	Define stored program concept	2	State definition + consequence
2	Von Neumann	Identify CPU components	2	List valid components inside CPU
3	Von Neumann	Explain limitation	2	State bottleneck + why it's a problem
4	Von Neumann	Explain benefit	2	Link to context (rewiring comparison)
5	Registers	Describe PC and MAR	4	What it holds + what happens to it
6	Buses	Explain bus directions	4	Reasoning about what travels where
7	Buses	Calculate addressable memory	2	Apply 2^n formula
8	ALU and CU	Describe purposes	4	Specific examples of operations
9	FDE interactions	Describe PC and MAR working together	3	Show flow between components

#	Topic	Question type	Marks	Key skill
10	Address bus	Calculate addressable memory (2024 style)	1	Apply 2^n formula
11	ALU/Accumulator	Explain relationship	2	Describe data flow
12	Diagram	Label buses and clock	3	Arrow directions matter

Quick reference

Key terms and definitions

Term	Definition (mark-scheme style)
Von Neumann architecture	Computer design where instructions and data are stored in the same memory and processed sequentially by a single CPU
Stored program concept	The idea that program instructions are stored in main memory alongside data, allowing programs to be changed without hardware modification
CPU	Central Processing Unit; the component that fetches, decodes and executes instructions
ALU	Arithmetic Logic Unit; performs arithmetic (add, subtract) and logical (AND, OR, compare) operations
Control Unit	Decodes instructions and sends control signals to coordinate the operation of other CPU components
Program Counter	Register that holds the memory address of the next instruction to be fetched
MAR	Memory Address Register; holds the address of data/instruction to be fetched or written
MDR	Memory Data Register; holds the data/instruction fetched from memory or waiting to be written
Accumulator	Register that stores the results of calculations performed by the ALU
System clock	Generates regular pulses to synchronise CPU operations; sends signals to the Control Unit
Address bus	Unidirectional bus that carries memory addresses from CPU to memory

Term	Definition (mark-scheme style)
Data bus	Bidirectional bus that carries data between CPU and memory/devices
Control bus	Bidirectional bus that carries control signals (read, write, clock, interrupt)

Powers of 2 reference

Power	Value	Use in CPU architecture
2^8	256	8-bit data bus = 256 possible values per byte
2^{10}	1,024	1 Ki (kibibyte base)
2^{16}	65,536	16-bit address bus = 64 Ki addresses
2^{20}	1,048,576	1 Mi (mebibyte base)
2^{32}	4,294,967,296	32-bit address bus = 4 Gi addresses (4 GiB max RAM)
2^{64}	≈18.4 quintillion	64-bit address bus = theoretical max (far more than current RAM sizes)

Bus direction summary

Bus	Direction	Memory trick
Address bus	CPU → Memory (one way)	"CPU **tells** memory where to look" — you don't ask for directions back
Data bus	CPU ↔ Memory (both ways)	"Data can be **read** from or **written** to memory"
Control bus	Bidirectional	"CPU sends commands, devices send interrupts back"

Typical pitfalls

Pitfall 1 — Confusing MAR and MDR.
Wrong: "The MAR holds the data fetched from memory."
Right: The MAR holds the **address**. The MDR holds the **data**. Memory **A**ddress Register = **A**ddress. Memory **D**ata Register = **D**ata.

Pitfall 2 — Saying the CU performs calculations.
Wrong: "The Control Unit performs addition and subtraction."

Right: The **ALU** performs calculations. The CU **controls and coordinates** — it sends signals, decodes instructions, and manages timing.

Pitfall 3 — Getting bus directions wrong.
Wrong: "The address bus is bidirectional."
Right: The address bus is **unidirectional** (CPU → memory). Only the data bus and control bus are bidirectional.

Pitfall 4 — Saying RAM is inside the CPU.
Wrong: "RAM is a component of the CPU."
Right: RAM is **separate** from the CPU, connected via buses. **Registers** are inside the CPU.

Pitfall 5 — Forgetting where the clock signal goes.
Wrong: "The clock is connected to the ALU" or drawing clock → ALU.
Right: The clock sends signals TO the **Control Unit**. Arrow direction: Clock → CU.

Pitfall 6 — Confusing fetch and execute.
Wrong: "The ALU fetches instructions from memory."
Right: The CU coordinates **fetching**. The ALU performs operations during **execute**. Different stages, different components.

Pitfall 7 — Confusing Hz units.
Wrong: "MHz means billions" or "GHz means millions."
Right: MHz = **million**. GHz = **billion**. A 3 GHz clock = 3 billion pulses per second.

Pitfall 8 — Wrong power of 2 for address bus.
Wrong: "A 32-bit address bus can address 32 memory locations."
Right: Number of addresses = 2^n. A 32-bit bus = 2^{32} = over 4 billion locations.

Pitfall 9 — Forgetting arrow direction on diagrams.
Wrong: Drawing arrows without direction, or clock pointing away from CU.
Right: Clock → CU (clock sends pulses TO the control unit). Address bus → Memory (one-way arrow).

Pitfall 10 — Vague answers about registers.
Wrong: "The PC stores information about the program."
Right: "The PC stores the **memory address of the next instruction to be fetched**." Be specific about what each register holds.

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.